

hwk2/sh2cpue.vhd

```

1  -----
2  --
3  -- Hitachi SH-2 CPU Entity Declaration
4  --
5  -- This is the entity declaration for the complete SH-2 CPU. The design
6  -- should implement this entity to make testing possible.
7  --
8  -- Revision History:
9  --      28 Apr 25  Glen George      Initial revision.
10 --      29 Apr 26  Simone Shevchuk  Initial revision.
11 --      01 May 26  Simone Shevchuk  Tri state busses and read/write, add addr_bit(1) as control input for IR
12 --      05 May 26  Simone Shevchuk  Add imm_sel control signal for selecting type of immediate value and add control
signals for NEG/NEGC
13 --      10 May 26  Simone Shevchuk  Sign extend databus for byte and word access
14 --      08 May 26  Simone Shevchuk  Update ALU inputs
15 --      11 May 26  Simone Shevchuk  Register the db value
16 --      12 May 26  Simone Shevchuk  Zero-extend s_db_hold for byte RMW ALU ops,
17 --                                  add alu_opb_bit7 mux for TAS.B bit 7 set.
18 --                                  Update input for PAU to be from reg_a for jump
19  -----
20
21
22 --
23 -- SH2_CPU
24 --
25 -- This is the complete entity declaration for the SH-2 CPU. It is used to
26 -- test the complete design.
27 --
28 -- Inputs:
29 --      Reset  - active low reset signal
30 --      NMI    - active falling edge non-maskable interrupt
31 --      INT    - active low maskable interrupt
32 --      clock  - the system clock
33 --
34 -- Outputs:
35 --      AB     - memory address bus (32 bits)
36 --      RE0    - first byte read signal, active low
37 --      RE1    - second byte read signal, active low
38 --      RE2    - third byte read signal, active low
39 --      RE3    - fourth byte read signal, active low
40 --      WE0    - first byte write signal, active low
41 --      WE1    - second byte write signal, active low
42 --      WE2    - third byte write signal, active low
43 --      WE3    - fourth byte write signal, active low
44 --
45 -- Inputs/Outputs:
46 --      DB     - memory data bus (32 bits)
47 --
48
49 library ieee;
50 use ieee.std_logic_1164.all;
51 --use ieee.std_logic_arith.all;
52 use ieee.numeric_std.all;
53 use work.InstructionConstants.all;
54
55 --library opcodes;
56 --use opcodes.opcodes.all;
57
58 entity SH2_CPU is
59
60     port (
61         Reset    : in    std_logic;           -- reset signal (active low)
62         NMI      : in    std_logic;           -- non-maskable interrupt signal (falling edge)
63         INT      : in    std_logic;           -- maskable interrupt signal (active low)
64         clock    : in    std_logic;           -- system clock

```

```

65     AB      : out    std_logic_vector(31 downto 0); -- memory address bus
66     RE0     : out    std_logic; -- first byte active low read enable
67     RE1     : out    std_logic; -- second byte active low read enable
68     RE2     : out    std_logic; -- third byte active low read enable
69     RE3     : out    std_logic; -- fourth byte active low read enable
70     WE0     : out    std_logic; -- first byte active low write enable
71     WE1     : out    std_logic; -- second byte active low write enable
72     WE2     : out    std_logic; -- third byte active low write enable
73     WE3     : out    std_logic; -- fourth byte active low write enable
74     DB      : inout  std_logic_vector(31 downto 0) -- memory data bus
75 );
76
77 end SH2_CPU;
78
79 architecture structural of SH2_CPU is
80
81     signal SR          : std_logic_vector(31 downto 0) := (others => '0'); -- status register to hold condition
codes and other status bits, T bit (s, m, q for multiply and divide no used currently)
82     signal ext_tbit    : std_logic_vector(31 downto 0); -- extended T bit value
83
84     signal s_IR : std_logic_vector(15 downto 0);
85
86     -- control signals from control unit to be sliced
87     signal s_reg_ALU_IN    : std_logic_vector(7 downto 0);
88     signal s_reg_MAU      : std_logic_vector(16 downto 0);
89     signal s_alu_IN       : std_logic_vector(1 downto 0);
90     signal s_alu_gen      : std_logic_vector(10 downto 0);
91     signal s_alu_sh2      : std_logic_vector(10 downto 0);
92     signal s_dau_controls : std_logic_vector(9 downto 0);
93     signal s_dau_write    : std_logic_vector(1 downto 0);
94     signal s_dau_out_sel  : std_logic_vector(1 downto 0);
95     signal s_pau_controls : std_logic_vector(5 downto 0);
96     signal s_RE          : std_logic_vector(3 downto 0);
97     signal s_WE          : std_logic_vector(3 downto 0);
98     signal s_addr_bus_sel : std_logic;
99     signal s_data_bus_sel : std_logic_vector(1 downto 0);
100    signal s_delay_slot   : std_logic;
101    signal s_sign_zero_sel : std_logic;
102    signal s_imm_sel      : std_logic_vector(1 downto 0);
103    signal s_tbit         : std_logic;
104    signal s_tbit_write   : std_logic;
105    signal s_alu_opa_zero : std_logic;
106    signal s_alu_opa_hold : std_logic;
107    signal s_alu_opb_bit7 : std_logic;
108    signal s_dau_load_data : std_logic;
109    signal s_db_hold      : std_logic_vector(31 downto 0);
110
111    -- data busses either driven or driving values to different blocks
112    signal s_pau_addr     : std_logic_vector(31 downto 0);
113    signal s_pr           : std_logic_vector(31 downto 0);
114    signal s_dau_addr     : std_logic_vector(31 downto 0);
115    signal s_dau_addr_wb  : std_logic_vector(31 downto 0);
116    signal s_dau_out      : std_logic_vector(31 downto 0); -- output from DAU that goes to register to write GBR/VBR
and also can be written to data bus for memory writes, can also output calculated address for MOVA
117    signal s_alu_result   : std_logic_vector(31 downto 0);
118    signal s_reg_in       : std_logic_vector(31 downto 0); -- value to write to register file
119    signal s_reg_in_sel   : std_logic_vector(2 downto 0); -- 000=ALU, 001=databus, 010=Tbit, 011=PR, 100=data_out
from DAU, 101=SR
120    signal s_reg_a        : std_logic_vector(31 downto 0); -- ALU operand A and used in PAU as source (sometimes)
and offset
121    signal s_reg_b        : std_logic_vector(31 downto 0); -- ALU Operand B
122    signal s_reg_ax_in    : std_logic_vector(31 downto 0); -- value to write to address register for address updates
from instructions
123    signal s_reg_r0       : std_logic_vector(31 downto 0); -- address bus 1 value for memory accesses offset, always
R0 for SH2
124    signal s_reg_a2       : std_logic_vector(31 downto 0); -- address bus 2 value for memory accesses basew register
for DAU
125    signal s_ext_sz_imm   : std_logic_vector(31 downto 0); -- sign/zero extended immediate

```

```

126     signal s_ext_imm      : std_logic_vector(31 downto 0); -- sign extended immediate for PAU and DAU, can be used
for offset for memory accesses and branch displacements
127     signal s_alu_opa      : std_logic_vector(31 downto 0); -- operand A for ALU
128     signal s_alu_opb      : std_logic_vector(31 downto 0); -- operand B for ALU
129     signal s_pau_datadata : std_logic_vector(31 downto 0); -- value to write to datadata bus for PAU, can be from
reg_a for JMP instructions or from DB for loading PR with LDC.L
130
131     begin
132
133         ext_tbit <= (31 downto 1 => '0') & SR(0); -- extend T bit to be able to write to register file and use as
ALU input
134         s_ext_imm <= (31 downto 4 => '0') & s_IR(3 downto 0)          when s_imm_sel = "01" else
135                     (31 downto 12 => s_IR(11)) & s_IR(11 downto 0) when s_imm_sel = "10" else
136                     (31 downto 8 => '0') & s_IR(7 downto 0)         when s_imm_sel = "11" else
137                     (31 downto 8 => s_IR(7)) & s_IR(7 downto 0); -- "00": 8-bit sign-extended
138
139         -- Address bus mux (tri-state during reset so TB can load program)
140         AB <= (others => 'Z') when reset = '0' else
141             s_pau_addr when s_addr_bus_sel = '0' else s_dau_addr;
142
143         -- Data bus mux (tri-state during reset and when not writing)
144         DB <= (others => 'Z') when (reset = '0' or s_WE = "1111") else
145             s_alu_result when (s_data_bus_sel = "00") else
146             s_dau_out     when (s_data_bus_sel = "01") else
147             SR            when (s_data_bus_sel = "10") else
148             s_pr;
149
150         -- RE/WE gated with clock
151         RE0 <= 'Z' when reset = '0' else (s_RE(0) or clock);
152         RE1 <= 'Z' when reset = '0' else (s_RE(1) or clock);
153         RE2 <= 'Z' when reset = '0' else (s_RE(2) or clock);
154         RE3 <= 'Z' when reset = '0' else (s_RE(3) or clock);
155         WE0 <= 'Z' when reset = '0' else (s_WE(0) or clock);
156         WE1 <= 'Z' when reset = '0' else (s_WE(1) or clock);
157         WE2 <= 'Z' when reset = '0' else (s_WE(2) or clock);
158         WE3 <= 'Z' when reset = '0' else (s_WE(3) or clock);
159
160         process(clock)
161         begin
162             if rising_edge(clock) then
163                 if reset = '0' then
164                     SR <= (31 downto 10 => '0') & "00" & "1111" & (3 downto 0 => '0');
165                 elsif s_tbit_write = '1' then
166                     SR(0) <= s_tbit; -- only update T bit for now since not implementing other status bits
167                 end if;
168             end if;
169         end process;
170
171
172         process(clock)
173         begin
174             if rising_edge(clock) then
175                 s_db_hold <= DB; -- clocked DB for read-write DB instructiiosn in ALU
176             end if;
177         end process;
178
179         -- Control Unit
180         ControlUnit : entity work.ControlSH2
181         port map (
182             DB          => DB,
183             addr_bit1    => s_pau_addr(1),
184             IR           => s_IR,
185             Tbit         => SR(0), -- T bit is bit 0 of status register
186             clock        => clock,
187             reset        => reset,
188             reg_ALU_IN   => s_reg_ALU_IN,
189             reg_MAU      => s_reg_MAU,
190             alu_IN       => s_alu_IN,

```

```

191     alu_gen          => s_alu_gen,
192     alu_sh2          => s_alu_sh2,
193     dau_controls     => s_dau_controls,
194     dau_write        => s_dau_write,
195     dau_out_sel      => s_dau_out_sel,
196     pau_controls     => s_pau_controls,
197     RE               => s_RE,
198     WE               => s_WE,
199     addr_bus_sel     => s_addr_bus_sel,
200     delay_slot       => s_delay_slot,
201     data_bus_sel     => s_data_bus_sel,
202     sign_zero_ALU_sel => s_sign_zero_sel,
203     tbit_write       => s_tbit_write,
204     imm_sel          => s_imm_sel,
205     alu_opa_zero     => s_alu_opa_zero,
206     alu_opa_hold     => s_alu_opa_hold,
207     alu_opb_bit7     => s_alu_opb_bit7,
208     dau_load_data    => s_dau_load_data
209 );
210
211
212 -- Register Array
213 s_reg_in_sel <= s_reg_ALU_IN(REG_IN_SRC_IDX downto REG_IN_SEL_IDX+1); -- select for which source to write to
register file
214 s_reg_in <= s_alu_result
215     (31 downto 8 => DB(7)) & DB(7 downto 0)
216     (31 downto 16 => DB(15)) & DB(15 downto 0)
217     DB
218     ext_tbit
219     s_pr
220     s_dau_out
221     SR
222     (others => 'X');
223
224 s_reg_ax_in <= s_dau_addr_wb; -- for instructions that write to address register for updates, value always
comes from data bus
225
226 RegArray : entity work.RegArrSH2
227 port map (
228     clock          => clock,
229     reg_in         => s_reg_in,
230     reg_in_sel     => to_integer(unsigned(s_reg_ALU_IN(REG_IN_SEL_IDX downto REG_STORE_IDX+1))),
231     reg_store      => s_REG_ALU_IN(REG_STORE_IDX),
232     reg_a_sel      => to_integer(unsigned(s_REG_MAU(REG_MAU_A_SEL_IDX downto REG_MAU_B_SEL_IDX+1))),
233     reg_b_sel      => to_integer(unsigned(s_REG_MAU(REG_MAU_B_SEL_IDX downto REG_MAU_AX_SEL_IDX+1))),
234     reg_ax_in      => s_reg_ax_in,
235     reg_ax_sel     => to_integer(unsigned(s_REG_MAU(REG_MAU_AX_SEL_IDX downto REG_MAU_AX_STORE_IDX+1))),
236     reg_ax_store   => s_REG_MAU(REG_MAU_AX_STORE_IDX),
237     reg_a2_sel     => to_integer(unsigned(s_REG_MAU(REG_MAU_A2_SEL_IDX downto 0))),
238     reg_a          => s_reg_a,
239     reg_b          => s_reg_b,
240     reg_a1         => s_reg_r0,
241     reg_a2         => s_reg_a2
242 );
243
244 -- PAU
245 -- mux datadata_src between reg_a (JMP address) and DB (LDS.L PR load)
246 s_pau_datadata <= DB when s_pau_controls(PAU_PR_SEL_IDX downto 0) = "11" else s_reg_a;
247
248 PAU : entity work.PAUSH2
249 port map (
250     clock          => clock,
251     reset          => reset,
252     datadata_src   => s_pau_datadata,
253     src_sel        => to_integer(unsigned(s_pau_controls(PAU_SRC_SEL_IDX downto PAU_OFFSET_SEL_IDX+1))),
254     addr_off       => s_ext_imm,
255     addr_reg       => s_reg_a,

```

```

256         offset_sel => to_integer(unsigned(s_pau_controls(PAU_OFFSET_SEL_IDX downto PAU_PR_SEL_IDX+1))),
257         pr_sel      => to_integer(unsigned(s_pau_controls(PAU_PR_SEL_IDX downto 0))),
258         delay_slot  => s_delay_slot,
259         address_out => s_pau_addr,
260         PR          => s_pr
261     );
262
263
264     -- DAU
265     DAU : entity work.DAUSH2
266     port map (
267         clock      => clock,
268         reset      => reset,
269         addr_PC    => s_pau_addr, -- for PC-relative accesses, use output of PAU since it has logic for
calculating next PC for branches and jumps
270         addr_reg   => s_reg_a2,
271         src_sel    => to_integer(unsigned(s_dau_controls(DAU_SRC_SEL_IDX downto DAU_SHIFT_SEL_IDX+1))),
272         shift_sel  => to_integer(unsigned(s_dau_controls(DAU_SHIFT_SEL_IDX downto
DAU_OFFSET_SEL_IDX+1))),
273         offset_ins => s_ext_imm,
274         offset_reg => s_reg_r0,
275         offset_sel => to_integer(unsigned(s_dau_controls(DAU_OFFSET_SEL_IDX downto
DAU_INC_DEC_SEL_IDX+1))),
276         inc_dec_sel => s_dau_controls(DAU_INC_DEC_SEL_IDX),
277         inc_dec_bit => to_integer(unsigned(s_dau_controls(DAU_INC_DEC_BIT_IDX downto DAU_PRE_POST_SEL_IDX-
+1))),
278         pre_post_sel => s_dau_controls(DAU_PRE_POST_SEL_IDX),
279         gbr_write   => s_dau_write(DAU_WRITE_GBR_IDX),
280         vbr_write   => s_dau_write(DAU_WRITE_VBR_IDX),
281         load_from_data => s_dau_load_data,
282         data_in     => DB,
283         data_out_sel => to_integer(unsigned(s_dau_out_sel)),
284         address_out => s_dau_addr,
285         addr_src_out => s_dau_addr_wb,
286         data_out    => s_dau_out
287     );
288
289     -- ALU
290
291     s_ext_sz_imm <= (31 downto 8 => s_IR(7)) & s_IR(7 downto 0) when s_sign_zero_sel = '1' else
292         (31 downto 8 => '0') & s_IR(7 downto 0); -- sign or zero extend immediate value from
instruction for ALU operations depending on instruction type
293         -- generally logical instructions zero extend
294
295     s_alu_opa <= (others => '0') when s_alu_opa_zero = '1' else
296         (31 downto 8 => '0') & s_db_hold(7 downto 0) when s_alu_opa_hold = '1' else
297         DB when s_alu_IN(ALU_SEL_A_IDX) = '0' else
298         s_reg_a when s_alu_IN(ALU_SEL_A_IDX) = '1' else
299         (others => 'X');
300
301     s_alu_opb <= x"00000080" when s_alu_opb_bit7 = '1' else
302         (others => '0') when (s_alu_gen(10) = s_alu_gen(9) and s_alu_gen(8) = s_alu_gen(7) and
s_alu_sh2(ALU_RES_SEL_IDX downto ALU_T_SEL_IDX+1) = "000") else -- if no ALU operation (pass through value), then
don't care value for operand B since it won't be used
303         s_reg_b when s_alu_IN(ALU_SEL_B_IDX) = '0' else
304         s_ext_sz_imm when s_alu_IN(ALU_SEL_B_IDX) = '1' else
305         (others => '0');
306
307     ALU : entity work.ALUSH2
308     port map (
309         alu_opa    => s_alu_opa,
310         alu_opb    => s_alu_opb,
311         c_in       => SR(0),
312         f_cmd      => s_alu_gen(ALU_F_CMD_IDX downto ALU_C_IN_CMD_IDX+1),
313         c_in_cmd   => s_alu_gen(ALU_C_IN_CMD_IDX downto ALU_S_CMD_IDX+1),
314         s_cmd      => s_alu_gen(ALU_S_CMD_IDX downto ALU_ALU_CMD_IDX+1),
315         alu_cmd    => s_alu_gen(ALU_ALU_CMD_IDX downto 0),
316         shift_sel  => to_integer(unsigned(s_alu_sh2(ALU_SHIFT_SEL_IDX downto ALU_EXTEND_SEL_IDX+1))),

```

```
317     extend_sel => s_alu_sh2(ALU_EXTEND_SEL_IDX downto ALU_RES_SEL_IDX+1),
318     res_sel => to_integer(unsigned(s_alu_sh2(ALU_RES_SEL_IDX downto ALU_T_SEL_IDX+1))),
319     t_sel => s_alu_sh2(ALU_T_SEL_IDX downto 0),
320     res => s_alu_result,
321     t_bit => s_tbit
322 );
323
324 end architecture structural;
325
```